PROMETEO

thePower
FP Oficial

Sistema de Gestión de Reservas de Pistas de Pádel

Desarrollo de Aplicaciones Multiplataformas / Online

Alumno: Marco Suárez Rodríguez

Tutora del TFG: Luz María Álvarez Moreno

PROMETEO

ÍNDICES

1. ABSTRACT **4**
1.1 Resumen 4
1.2 Abstract 4
2. JUSTIFICACIÓN DEL PROYECTO **5**
2.1 Motivación del proyecto 5
2.2 Estado de la cuestión 5
2.3 Comparativa con aplicaciones similares 6
3. INTRODUCCIÓN **8**
3.1 Requisitos principales del sistema 9
4. OBJETIVOS **10**
5. DESCRIPCIÓN **16**
5.1 Arquitectura de la Solución 16
5.2 Casos de uso 17
6. DISEÑOS **21**
6.1 Diagrama Entidad-Relación 21
6.2 Diagrama de la Base de Datos 22
6.3 Diagrama de Clases 23
7. TECNOLOGÍAS **25**
7.1 MySQL Workbench 25
7.2 Java 26
7.3 JDBC 27
7.4 Java Swing 28
7.5 Draw.io 29
7.6 Visual Studio Code 30
7.7 Google Docs 31
9. TRABAJOS FUTUROS **33**
10. CONCLUSIONES **34**

1. ABSTRACT

1.1 Resumen

El presente proyecto tiene como objetivo el desarrollo de un sistema de gestión de reservas para pistas de pádel, diseñado para facilitar la administración de usuarios, pistas deportivas y reservas dentro de una instalación deportiva.

La aplicación se ha desarrollado utilizando el lenguaje de programación Java y una base de datos relacional implementada en MySQL. Para la comunicación entre la aplicación y la base de datos se ha utilizado JDBC, lo que permite gestionar la información almacenada mediante operaciones de inserción, consulta, actualización y eliminación de datos. El sistema se basa en una arquitectura modular que separa los modelos de datos, el acceso a la base de datos y la lógica de la aplicación. Para ello se han implementado clases modelo que representan las entidades principales del sistema (usuarios, pistas, reservas, tarifas y pagos), así como clases DAO (Data Access Object) encargadas de gestionar las operaciones sobre la base de datos.

1.2 Abstract

This project aims to develop a reservation management system for padel courts, designed to facilitate the administration of users, sports courts, and reservations within a sports facility.

The application has been developed using the Java programming language and a relational database implemented in MySQL. The communication between the application and the database is handled through JDBC, which allows the management of stored information through basic database operations such as insertion, querying, updating, and deletion. The system follows a modular architecture that separates data models, database access, and application logic. For this purpose, model classes have been implemented to represent the main entities of the system (users, courts, reservations, fee and payments), along with DAO (Data Access Object) classes responsible for managing interactions with the database.

2. JUSTIFICACIÓN DEL PROYECTO

2.1 Motivación del proyecto

La gestión de reservas en instalaciones deportivas es una tarea habitual en clubes, centros deportivos y polideportivos municipales. En muchos casos, esta gestión se realiza mediante herramientas poco especializadas, como hojas de cálculo, agendas manuales o sistemas informáticos muy básicos. Este tipo de soluciones puede generar problemas de organización, dificultades para consultar la disponibilidad de las pistas y errores en la gestión de las reservas.

El crecimiento del número de usuarios en deportes como el pádel ha incrementado la necesidad de contar con sistemas que permitan gestionar de forma eficiente las reservas de pistas. Un sistema informático diseñado específicamente para este propósito permite centralizar la información, facilitar la consulta de disponibilidad y mejorar la organización de las instalaciones deportivas.

La motivación principal de este proyecto es desarrollar una aplicación que permita gestionar de forma sencilla la información relacionada con usuarios, pistas de pádel y reservas, utilizando tecnologías estudiadas durante el ciclo formativo de Desarrollo de Aplicaciones Multiplataforma. De este modo, el proyecto no solo busca resolver un problema práctico de gestión, sino también servir como aplicación real de los conocimientos adquiridos en programación, bases de datos y acceso a datos.

2.2 Estado de la cuestión

Actualmente existen diferentes aplicaciones y plataformas que permiten gestionar reservas de instalaciones deportivas. Muchas de estas soluciones están orientadas a clubes deportivos o centros con gran número de usuarios, e incluyen funcionalidades avanzadas como gestión de pagos, control de socios, integración con aplicaciones móviles o sistemas de notificaciones.

Entre las características más comunes de este tipo de aplicaciones se encuentran:

- Gestión de usuarios registrados.
- Consulta de disponibilidad de pistas.
- Reserva de instalaciones deportivas.
- Gestión de pagos asociados a las reservas.
- Historial de reservas realizadas.

Sin embargo, muchas de estas plataformas son soluciones comerciales complejas que requieren una configuración avanzada o están orientadas a entornos empresariales. Por ello, para fines educativos resulta interesante desarrollar una versión simplificada que permita comprender el funcionamiento interno de este tipo de sistemas.

El proyecto desarrollado propone una solución básica pero funcional que permite gestionar usuarios, pistas, pagos, tarifas y reservas mediante una base de datos relacional conectada a una aplicación Java.

2.3 Comparativa con aplicaciones similares

Existen diversas aplicaciones que ofrecen servicios de gestión de reservas deportivas, tanto en formato web como en aplicaciones móviles. Algunas de las más conocidas permiten realizar reservas online, consultar la disponibilidad de instalaciones o gestionar pagos asociados a las reservas.

No obstante, estas soluciones suelen estar diseñadas para entornos comerciales y presentan una gran cantidad de funcionalidades que exceden el alcance de este proyecto. El sistema propuesto en este trabajo tiene un enfoque principalmente educativo y busca implementar las funcionalidades esenciales necesarias para comprender el funcionamiento de un sistema de gestión de reservas.

A diferencia de las aplicaciones comerciales existentes, este proyecto se centra en el desarrollo desde cero de la base de datos y de la lógica de acceso a datos utilizando Java y MySQL. Esto permite analizar en detalle la estructura del sistema, la gestión de la información y la implementación de operaciones sobre la base de datos.

El resultado es una aplicación sencilla pero estructurada que permite gestionar reservas de pistas de pádel y que puede ampliarse en el futuro con nuevas funcionalidades, como la gestión de tarifas, pagos o una interfaz gráfica de usuario.

3. INTRODUCCIÓN

El presente proyecto consiste en el desarrollo de un sistema de gestión de reservas para pistas de pádel. El objetivo principal de la aplicación es facilitar la administración de usuarios, pistas deportivas, pagos, tarifas y reservas dentro de una instalación deportiva, permitiendo organizar de forma eficiente la disponibilidad y el uso de las pistas.

En muchas instalaciones deportivas, la gestión de reservas se realiza mediante métodos manuales o mediante herramientas poco especializadas. Esto puede generar dificultades para consultar la disponibilidad de las pistas, errores en la organización de horarios o problemas en la gestión de la información de los usuarios. Un sistema informático específico permite centralizar toda esta información y mejorar la gestión de las reservas.

La aplicación desarrollada permite registrar usuarios, gestionar las pistas disponibles y realizar reservas asociadas a fechas y horarios concretos, teniendo en cuenta tarifas y métodos de pago. Toda la información se almacena en una base de datos relacional, lo que permite mantener los datos organizados y garantizar la integridad de la información.

El sistema se ha desarrollado utilizando el lenguaje de programación Java y una base de datos MySQL. Para la comunicación entre la aplicación y la base de datos se utiliza JDBC, lo que permite realizar operaciones de inserción, consulta, actualización y eliminación de datos. La estructura del proyecto sigue una arquitectura basada en modelos y clases de acceso a datos (DAO), lo que facilita la organización del código y la ampliación futura del sistema.

Este proyecto tiene también un enfoque formativo, ya que permite aplicar de forma práctica los conocimientos adquiridos durante el ciclo formativo de Desarrollo de Aplicaciones Multiplataforma, especialmente en áreas como programación orientada a objetos, diseño de bases de datos y acceso a datos desde aplicaciones Java.

3.1 Requisitos principales del sistema

Para que el sistema cumpla su función correctamente, debe satisfacer una serie de requisitos funcionales básicos y técnicos:

- Permitir el registro y almacenamiento de usuarios dentro del sistema.
- Gestionar la información de las pistas de pádel disponibles.
- Permitir la creación de reservas asociadas a un usuario y a una pista concreta.
- Gestionar las tarifas aplicables a las reservas y registrar los pagos realizados por los usuarios según el tipo de pista, el día de la semana y la franja horaria seleccionada.
- Consultar la información almacenada en la base de datos. Actualizar los datos existentes cuando sea necesario.
- Eliminar registros que ya no sean necesarios.
- Utilizar una base de datos relacional para el almacenamiento de la información.
- Implementar la conexión entre la aplicación Java y la base de datos mediante JDBC.
- Organizar el código utilizando una estructura modular basada en clases modelo y clases DAO.
- Garantizar la correcta gestión de los datos mediante operaciones CRUD (Create, Read, Update, Delete).

4. OBJETIVOS

Para definir de forma estructurada los objetivos del sistema, se utiliza el modelo RFTP, que permite organizar los requisitos, funciones, tareas y pruebas necesarias para el desarrollo del proyecto.

R01 – El sistema debe permitir gestionar usuarios dentro de la aplicación.

R01F01 – El sistema debe permitir registrar usuarios en la base de datos.

R01F01T01 – Diseñar la tabla usuario en la base de datos.

R01F01T01P01 – Verificar que la tabla se crea correctamente en MySQL.

R01F01T02 – Crear la clase modelo Usuario en Java.

R01F01T02P01 – Comprobar que la clase permite almacenar correctamente los atributos del usuario.

R01F01T03 – Implementar el DAO UsuarioDAO para gestionar las operaciones sobre la tabla.

R01F01T03P01 – Insertar un usuario de prueba en la base de datos.

R01F02 – El sistema debe permitir consultar los usuarios registrados.

R01F02T01 – Implementar el método obtenerUsuarios() en el DAO.

R01F02T01P01 – Ejecutar la consulta desde la aplicación y mostrar los resultados por consola.

R01F03 – El sistema debe permitir modificar la información de un usuario.

R01F03T01 – Implementar el método de actualización en el DAO.

R01F03T01P01 – Modificar un usuario existente y comprobar el cambio en la base de datos.

R01F04 – El sistema debe permitir eliminar usuarios del sistema.

R01F04T01 – Implementar el método de eliminación en el DAO.

R01F04T01P01 – Eliminar un usuario de prueba y verificar su eliminación en la base de datos.

R02 – El sistema debe permitir gestionar las pistas de pádel.

R02F01 – El sistema debe almacenar información sobre las pistas disponibles.

R02F01T01 – Diseñar la tabla pista en la base de datos.

R02F01T01P01 – Comprobar que la tabla se crea correctamente.

R02F02 – El sistema debe permitir registrar pistas en el sistema.

R02F02T01 – Crear la clase modelo Pista.

R02F02T01P01 – Verificar que los atributos se almacenan correctamente.

R02F02T02 – Implementar el DAO PistaDAO.

R02F02T02P01 – Insertar una pista de prueba en la base de datos.

R03 – El sistema debe permitir gestionar reservas.

R03F01 – El sistema debe permitir crear reservas.

R03F01T01 – Diseñar la tabla reserva en la base de datos.

R03F01T01P01 – Verificar que la tabla se crea correctamente.

R03F01T02 – Crear la clase modelo Reserva.

R03F01T02P01 – Comprobar el correcto almacenamiento de los datos.

R03F01T03 – Implementar el DAO ReservaDAO.

R03F01T03P01 – Insertar una reserva de prueba en la base de datos.

R03F02 – El sistema debe permitir consultar las reservas existentes.

R03F02T01 – Implementar el método para consultar reservas.

R03F02T01P01 – Mostrar las reservas almacenadas en la base de datos.

R03F03 – El sistema debe permitir modificar reservas.

R03F03T01 – Implementar el método de actualización de reservas.

R03F03T01P01 – Modificar una reserva existente y comprobar el resultado.

R03F04 – El sistema debe permitir eliminar reservas.

R03F04T01 – Implementar el método de eliminación de reservas.

R03F04T01P01 – Eliminar una reserva de prueba.

R04 – El sistema debe gestionar las tarifas aplicables a las reservas en función del tipo de pista, el día de la semana y la franja horaria.

R04F01 – El sistema debe identificar el tipo de pista reservada.

R04F01T01 – Consultar el tipo de pista (cubierta o exterior) en la base de datos.

R04F01T01P01 – Recuperar el campo tipo de la tabla pista.

R04F02 – El sistema debe identificar si la reserva se realiza en día laborable o fin de semana.

R04F02T01 – Determinar el tipo de día a partir de la fecha de la reserva.

R04F02T01P01 – Comprobar que el sistema asigna correctamente el valor "Laborable" o "FinDeSemana".

R04F03 – El sistema debe identificar la franja horaria de la reserva.

R04F03T01 – Determinar si la reserva pertenece a la franja de mañana o tarde según la hora seleccionada.

R04F03T01P01 – Verificar que el sistema asigna correctamente la franja horaria.

R04F04 – El sistema debe obtener el precio correspondiente a la reserva.

R04F04T01 – Consultar la tabla tarifa para obtener el precio según los criterios anteriores.

R04F04T01P01 – Verificar que el precio obtenido coincide con el definido en la base de datos.

R05 – El sistema debe registrar el pago asociado a cada reserva.

R05F01 – El sistema debe permitir registrar un pago para una reserva.

R05F01T01 – Crear un registro en la tabla pago asociado a una reserva.

R05F01T01P01 – Introducir un pago de prueba y verificar que se almacena correctamente.

R05F02 – El sistema debe permitir indicar el método de pago utilizado.

R05F02T01 – Registrar el método de pago (Tarjeta, Efectivo o Bizum).

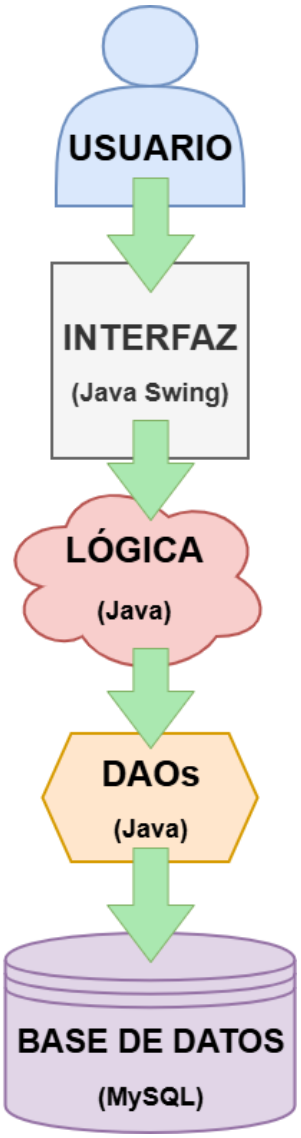
R05F02T01P01 – Comprobar que el método se guarda correctamente en la base de datos.

5. DESCRIPCIÓN

5.1 Arquitectura de la Solución

El sistema desarrollado sigue una arquitectura por capas, separando claramente la interfaz de usuario, la lógica de negocio y el acceso a datos. Esta organización permite un diseño modular, facilitando el mantenimiento, la escalabilidad y la reutilización del código.

La aplicación se ha implementado en Java utilizando Java Swing para la interfaz gráfica y JDBC para la conexión con la base de datos MySQL.



Las capas principales del sistema son:

Capa de Presentación: Encargada de la interacción con el usuario. Está compuesta por las clases VistaReserva, VistaListaReserva y VistaEditarReserva, desarrolladas con Java Swing. Permite crear, visualizar, editar y eliminar reservas mediante una interfaz gráfica.

Capa de Lógica de Negocio: Esta capa se encarga de validar datos y aplicar las reglas del negocio antes de acceder a la base de datos. Contiene la lógica del sistema como el cálculo automático de tarifas o el control de solapamiento de reservas.

Capa de Acceso a Datos: Implementada mediante clases DAO (ReservaDAO, UsuarioDAO, PistaDAO, etc.), que se encargan de realizar las operaciones CRUD sobre la base de datos utilizando JDBC.

Capa de Datos: Base de datos MySQL compuesta por las tablas: usuario, pista, reserva, tarifa y pago, relacionadas entre sí mediante claves foráneas.

5.2 Casos de uso

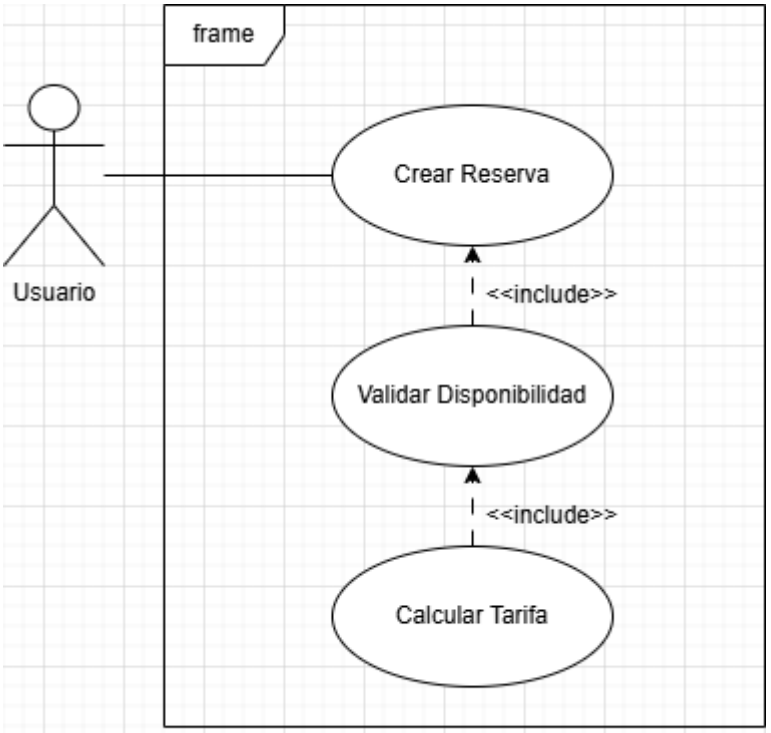


Ilustración 1: caso de uso Crear Reserva

DESCRIPCIÓN: Permite al usuario crear una nueva reserva de una pista de pádel introduciendo los datos necesarios.	
PRECONDICIONES: El usuario introduce correctamente los datos (usuario, pista, fecha y hora).	**POSTCONDICIONES:** La reserva queda registrada en la base de datos si no hay conflictos de horario.
DATOS ENTRADA: ID usuario, ID pista, fecha, hora inicio, hora fin, tipo de día, franja horaria.	**DATOS SALIDA:** Confirmación de reserva creada o mensaje de error.
TABLAS: Reserva, pista, tarifa.	**CLASES:** VistaReserva, ReservaDAO, ManejoTarifa, Reserva
INTERFACES: Interfaz gráfica (Java Swing)	

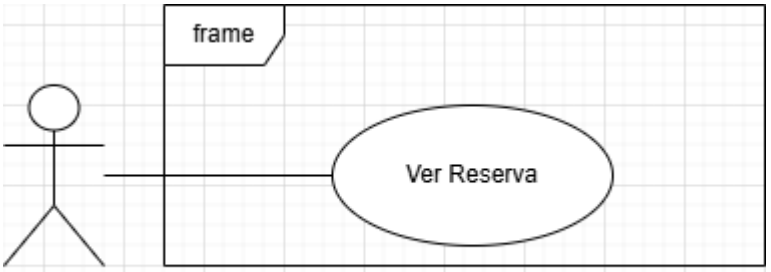


Ilustración 2: caso de uso VerReserva

DESCRIPCIÓN: Permite al usuario visualizar todas las reservas almacenadas en el sistema.	
PRECONDICIONES: Existen reservas en la base de datos.	**POSTCONDICIONES:** Se muestra una tabla con las reservas.
DATOS ENTRADA: Ninguno.	**DATOS SALIDA:** Lista de reservas.
TABLAS: Reserva.	**CLASES:** VistaListaReserva, ReservaDAO.
INTERFACES: Interfaz gráfica (Java Swing).	

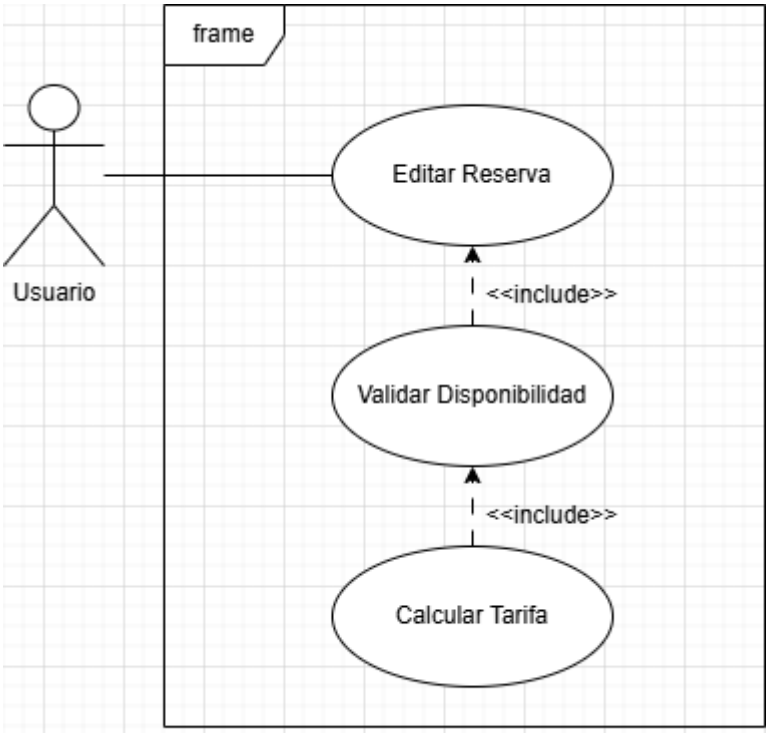


Ilustración 3: caso de uso Editar Reserva

DESCRIPCIÓN: Permite modificar los datos de una reserva existente.	
PRECONDICIONES: La reserva existe en la base de datos.	**POSTCONDICIONES:** La reserva se actualiza correctamente si no hay solapamientos.
DATOS ENTRADA: ID reserva, usuario, pista, fecha, horas.	**DATOS SALIDA:** Confirmación o error.
TABLAS: Reserva, pista, tarifa.	**CLASES:** VistaEditarReserva, ReservaDAO, ManejoTarifa.
INTERFACES: Interfaz gráfica (Java Swing).	

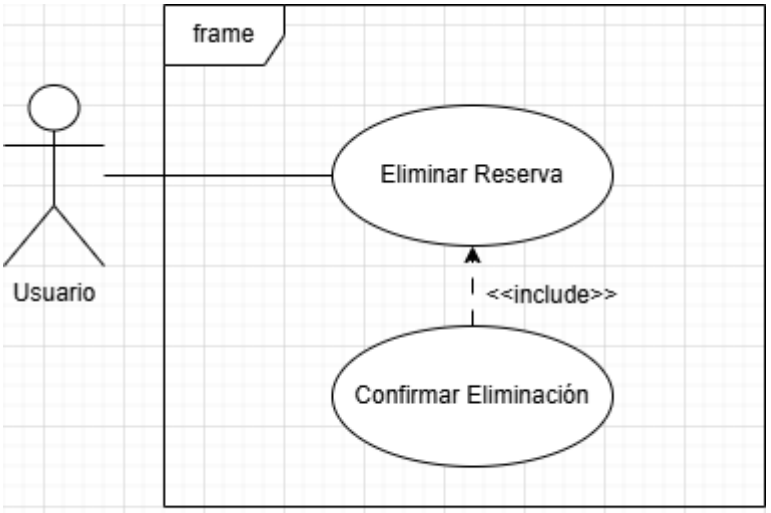
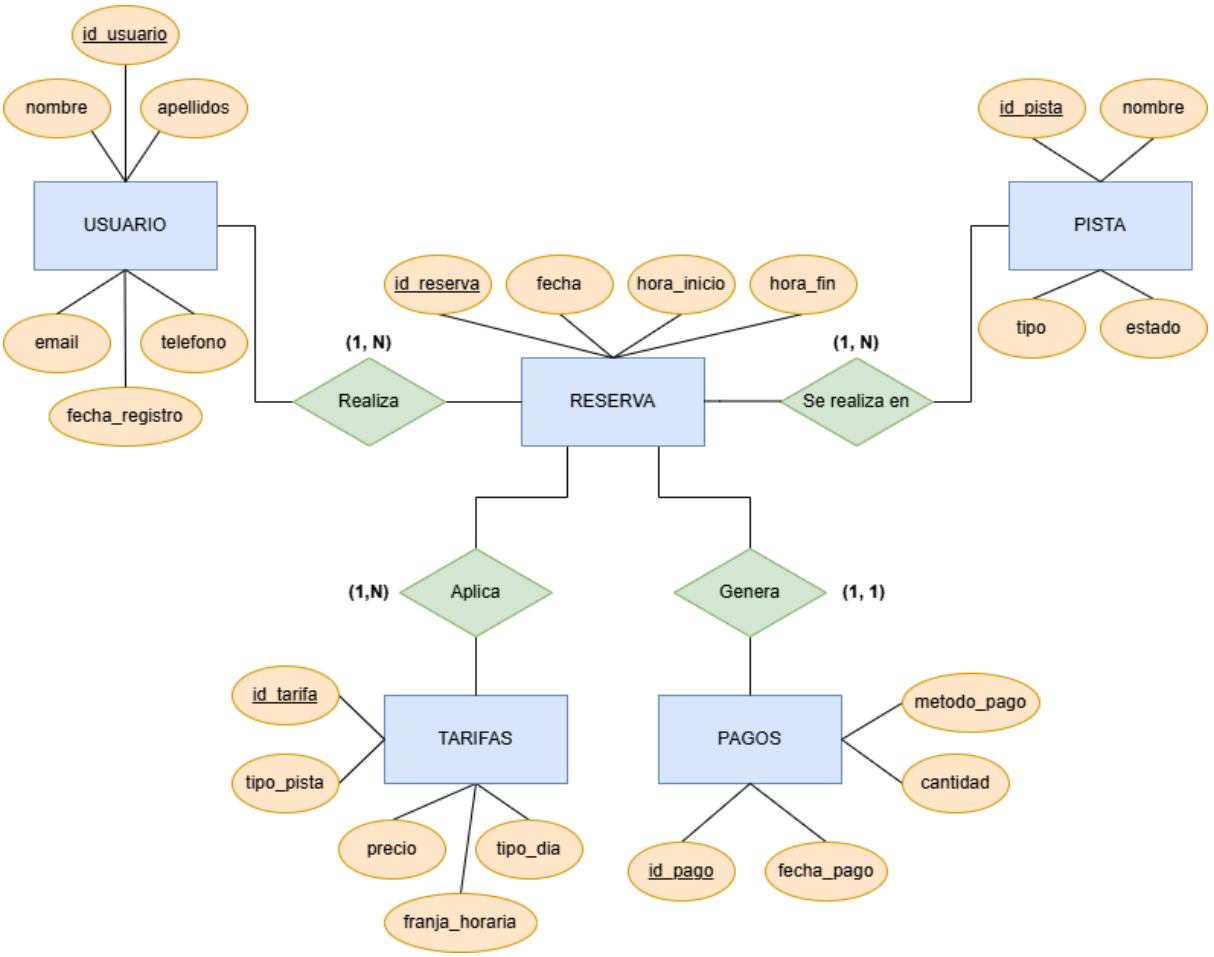


Ilustración 4: caso de uso Eliminar Reserva

DESCRIPCIÓN: Permite eliminar una reserva existente del sistema.	
PRECONDICIONES: La reserva existe en la base de datos.	**POSTCONDICIONES:** La reserva es eliminada de la base de datos.
DATOS ENTRADA: ID reserva	**DATOS SALIDA:** Confirmación de eliminación.
TABLAS: Reserva.	**CLASES:** VistaListaReserva, ReservaDAO.
INTERFACES: Interfaz gráfica (Java Swing).	

6. DISEÑOS

6.1 Diagrama Entidad-Relación



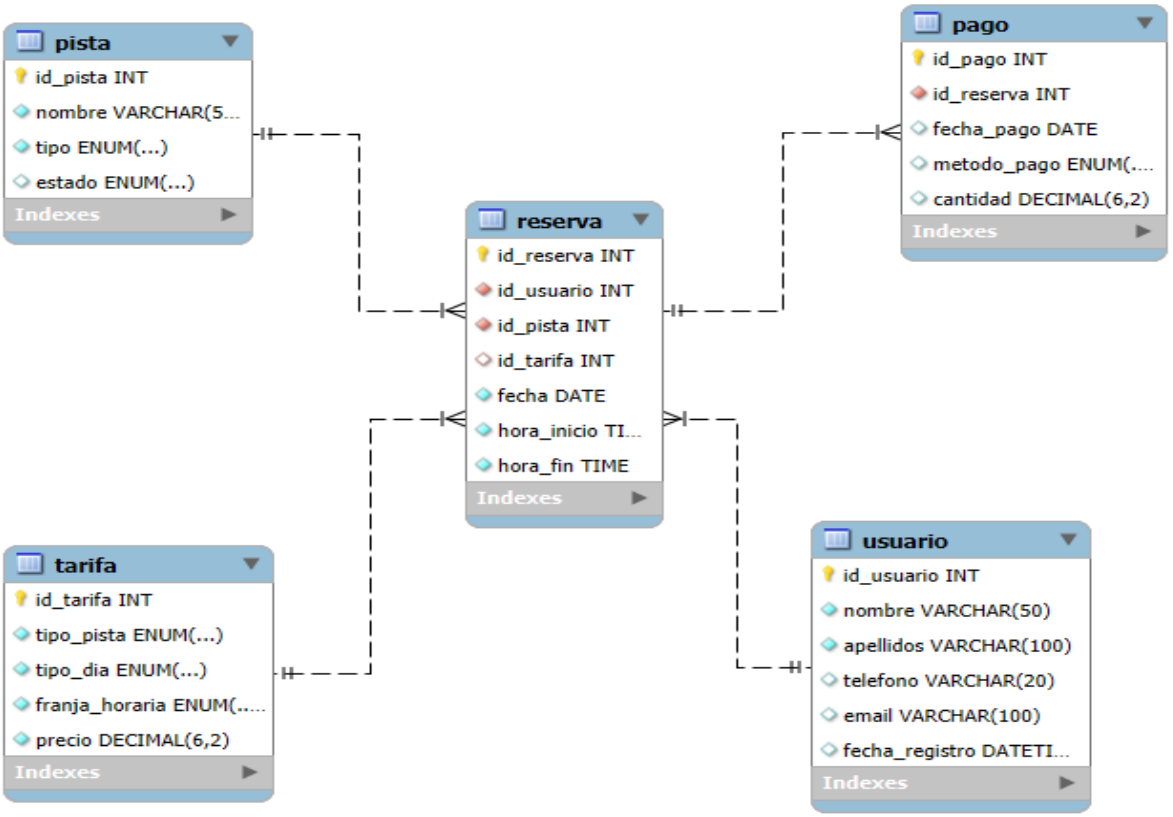
El modelo entidad-relación representa la estructura conceptual de la base de datos del sistema. En este modelo se identifican las entidades principales del sistema: Usuario, Pista, Reserva, Tarifa y Pago.

La entidad Usuario representa a los clientes registrados en el sistema, mientras que Pista almacena la información de las pistas disponibles en el club.

La entidad Reserva actúa como elemento central del sistema, ya que relaciona a los usuarios con las pistas en una fecha y horario determinados.

La entidad Tarifa permite determinar el precio de la reserva en función del tipo de pista, el tipo de día y la franja horaria. Finalmente, la entidad Pago registra la información relativa al pago asociado a cada reserva realizada.

6.2 Diagrama de la Base de Datos



La base de datos del sistema ha sido diseñada siguiendo un modelo relacional que permite gestionar la información necesaria para el funcionamiento del sistema de reservas de pistas de pádel. El sistema se compone de cinco tablas principales: usuario, pista, reserva, tarifa y pago.

La tabla usuario almacena la información de los clientes registrados en el sistema.

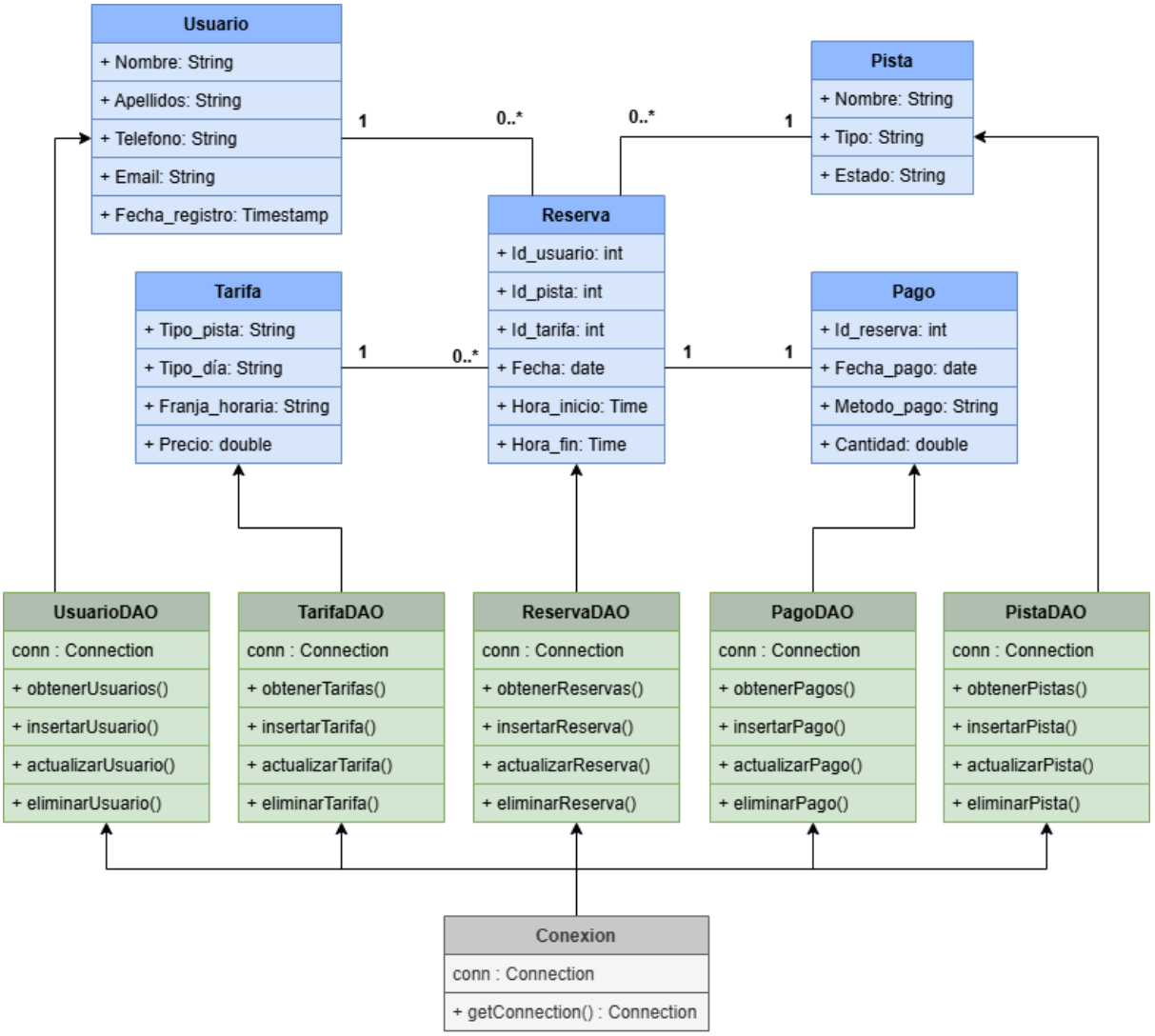
La tabla pista contiene los datos de las pistas disponibles en el club, incluyendo su tipo y estado.

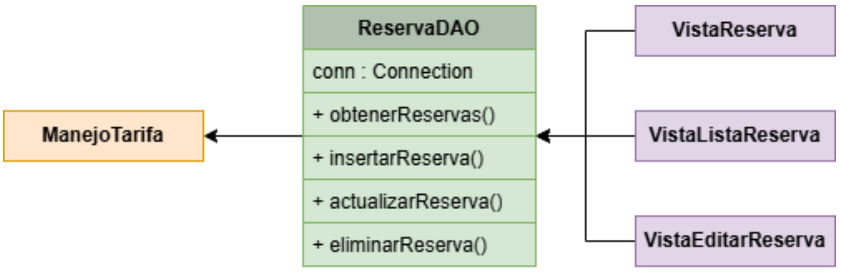
La tabla reserva gestiona las reservas realizadas por los usuarios, relacionando cada reserva con una pista y una tarifa aplicable.

La tabla tarifa permite definir los precios en función del tipo de pista, el tipo de día y la franja horaria.

Finalmente, la tabla pago registra los pagos asociados a las reservas realizadas.

6.3 Diagrama de Clases





El sistema ha sido desarrollado utilizando la programación orientada a objetos. Para ello se han definido diferentes clases que representan las entidades principales del sistema y las operaciones necesarias para gestionar la información almacenada en la base de datos.

Las clases Usuario, Pista, Reserva, Tarifa y Pago representan el modelo de datos del sistema, ya que almacenan los atributos correspondientes a cada entidad de la base de datos.

Para gestionar el acceso a los datos se ha utilizado el patrón DAO (Data Access Object), implementado mediante las clases UsuarioDAO, PistaDAO, ReservaDAO, TarifaDAO y PagoDAO. Estas clases encapsulan las operaciones de acceso a la base de datos, permitiendo realizar operaciones CRUD (crear, consultar, modificar y eliminar) sobre las diferentes tablas.

Además, se ha incorporado una clase de lógica denominada ManejoTarifa, cuya función es calcular automáticamente la tarifa correspondiente a una reserva en función del tipo de pista, el tipo de día y la franja horaria.

En cuanto a la interfaz gráfica, se han desarrollado las clases VistaReserva (que permite la creación de nuevas reservas), VistaListaReserva (que muestra las reservas existentes en forma de tabla, facilitando su consulta) y VistaEditarReserva (que permite modificar los datos de una reserva ya existente).

Estas clases de vista se comunican con las clases DAO para ejecutar las operaciones correspondientes, estableciendo así la conexión entre la interfaz de usuario y la lógica del sistema.

Finalmente, la clase Conexion se encarga de gestionar la conexión con la base de datos MySQL utilizada por la aplicación.

7. TECNOLOGÍAS

7.1 MySQL Workbench

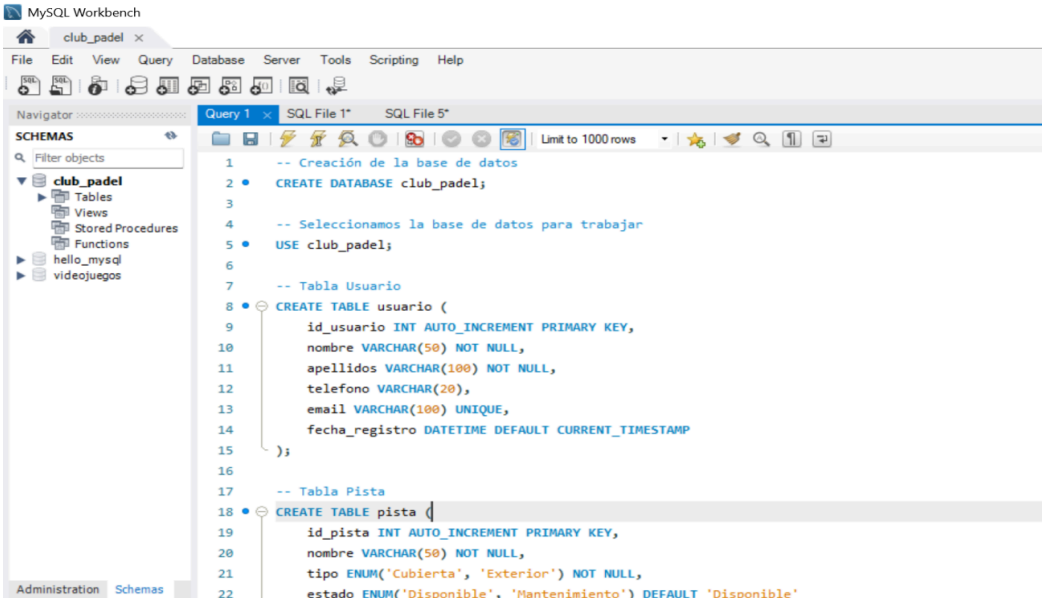
Para el almacenamiento de la información se ha utilizado el sistema de gestión de bases de datos MySQL, uno de los motores de bases de datos relacionales más extendidos en el desarrollo de aplicaciones.

Para el diseño de la base de datos se ha utilizado la herramienta MySQL Workbench, que permite trabajar de forma visual con bases de datos dentro del propio MySQL.

Esta tecnología se ha utilizado para crear la base de datos del proyecto, definir las tablas y sus relaciones, ejecutar consultas SQL, insertar datos de prueba y generar el diagrama de la base de datos utilizado en la documentación del proyecto.

La estructura de la base de datos está compuesta por cinco tablas principales: usuario, pago, reserva, pista y reserva.

Estas tablas se encuentran relacionadas mediante claves primarias y claves foráneas, lo que garantiza la integridad de los datos y permite establecer las relaciones necesarias entre las diferentes entidades del sistema. El uso de esta herramienta facilita el desarrollo y la visualización de la estructura de la base de datos.



7.2 Java

El desarrollo de la lógica de la aplicación se ha realizado utilizando el lenguaje de programación Java, uno de los lenguajes más utilizados en el desarrollo de aplicaciones empresariales y sistemas basados en objetos.

Java permite aplicar el paradigma de programación orientada a objetos, lo que facilita la organización del código mediante clases, atributos y métodos. En el proyecto se han creado diferentes clases que representan las entidades principales del sistema, como Usuario, Pista, Reserva, Tarifa y Pago.

Además, se ha implementado el patrón DAO (Data Access Object) para separar la lógica de acceso a datos del resto de la aplicación. Para ello se han desarrollado las clases UsuarioDAO, PistaDAO, ReservaDAO, TarifaDAO y PagoDAO, que permiten realizar operaciones de inserción, consulta, modificación y eliminación de registros en la base de datos.

Este enfoque permite mantener un código más organizado, modular y fácil de mantener.

UsuarioDAO.java

Java > dao > UsuarioDAO.java > UsuarioDAO > obtenerUsuarios()

```java
package dao;

import modelo.Usuario;
import conexion.conexion;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class UsuarioDAO {

    public List<Usuario> obtenerUsuarios() throws SQLException {

        List<Usuario> usuarios = new ArrayList<>();
        String sql = "SELECT * FROM usuario";

        try (Connection conn = conexion.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {

                Usuario u = new Usuario(
                    rs.getInt(columnLabel: "id_usuario"),
                    rs.getString(columnLabel: "nombre"),
                    rs.getString(columnLabel: "apellidos"),
                    rs.getString(columnLabel: "telefono"),
                    rs.getString(columnLabel: "email"),
                    rs.getTimestamp(columnLabel: "fecha_registro")
                );

                usuarios.add(u);
            }
        }

        return usuarios;
    }


    public void insertarUsuario(Usuario usuario) throws SQLException {
```

7.3 JDBC

Para permitir la comunicación entre la aplicación Java y la base de datos MySQL se ha utilizado JDBC (Java Database Connectivity).

JDBC es una API que permite ejecutar consultas SQL desde aplicaciones Java y gestionar los resultados obtenidos de la base de datos.

En el proyecto se ha implementado una clase denominada Conexion, encargada de establecer la conexión con la base de datos. A partir de esta conexión, las clases DAO pueden ejecutar consultas SQL mediante objetos como Connection, Statement, PreparedStatement y ResultSet.

Esto permite realizar operaciones de lectura y escritura sobre la base de datos desde la aplicación Java.

```
conexion.java
Java > conexion > conexion.java > conexion > PASS
1   package conexion;
2
3   import java.sql.Connection;
4   import java.sql.DriverManager;
5   import java.sql.SQLException;
6
7   public class conexion {
8       private static final String URL = "jdbc:mysql://localhost:3306/club_padel";
9       private static final String USER = "[redacted]";
10      private static final String PASS = "[redacted]";
11
12      public static Connection getConnection() throws SQLException {
13          return DriverManager.getConnection(URL, USER, PASS);
14      }
15  }
16
```

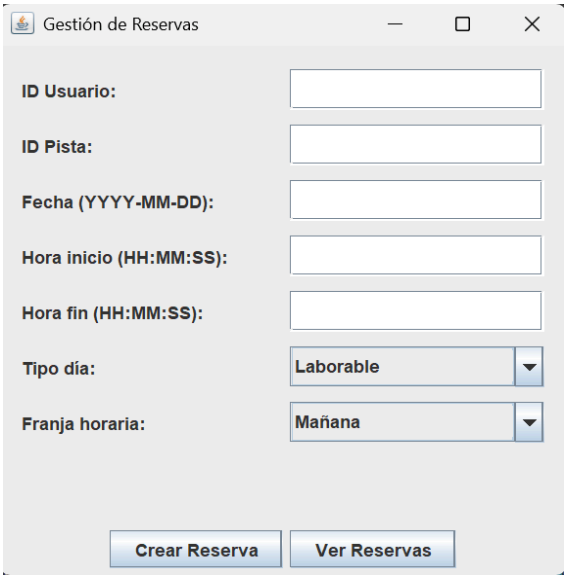
7.4 Java Swing

Java Swing es una biblioteca gráfica incluida en Java que permite desarrollar interfaces de usuario (GUI) para aplicaciones de escritorio. Proporciona componentes visuales como ventanas, botones, etiquetas, campos de texto y tablas, que permiten la interacción entre el usuario y la aplicación. Swing forma parte del paquete javax.swing y se caracteriza por ser independiente del sistema operativo, lo que permite que las aplicaciones desarrolladas funcionen en diferentes plataformas sin necesidad de modificaciones.

En este proyecto, Java Swing se ha utilizado para desarrollar la interfaz gráfica del sistema de gestión de reservas de pistas de pádel.

Se han implementado varias ventanas principales:

- Una ventana para crear reservas, donde el usuario introduce los datos necesarios como el identificador del usuario, la pista, la fecha y el horario.
- Una ventana para visualizar las reservas existentes en forma de tabla.
- Una ventana para editar reservas, permitiendo modificar sus datos.



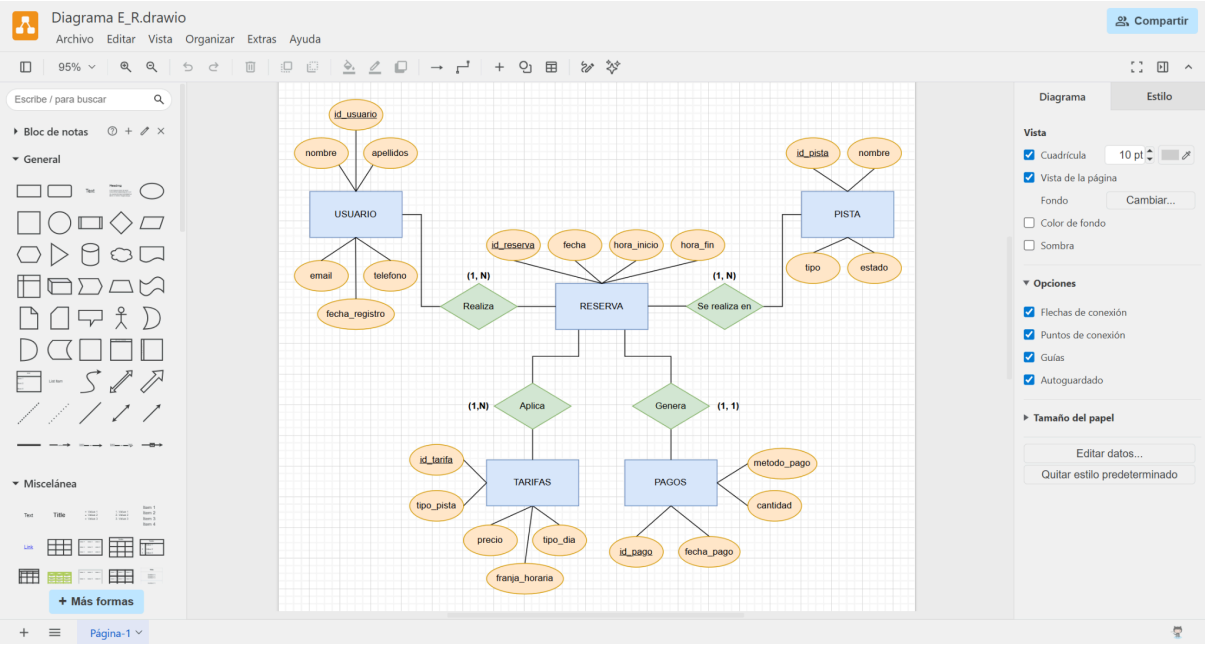
A través de botones y campos de entrada, el usuario puede interactuar con la aplicación de forma sencilla e intuitiva. Estas acciones están conectadas con la lógica del programa mediante eventos (ActionListener), que permiten ejecutar operaciones como crear, actualizar o eliminar reservas.

La interfaz se conecta con la capa de acceso a datos (DAO), permitiendo que las acciones realizadas por el usuario se reflejen directamente en la base de datos.

7.5 Draw.io

Draw.io es una herramienta de diseño de diagramas que permite crear representaciones gráficas de sistemas, como diagramas de clases, diagramas entidad-relación o diagramas de flujo. Se caracteriza por ser fácil de usar y por permitir exportar los diagramas en diferentes formatos, como imagen o PDF.

En este proyecto, Draw.io se ha utilizado para la creación de los distintos diagramas necesarios para la documentación, como el diagrama entidad-relación, el diagrama de clases, el diagrama de arquitectura y los diagramas de casos de uso. Gracias a esta herramienta, se ha podido representar de forma visual la estructura y el funcionamiento del sistema.



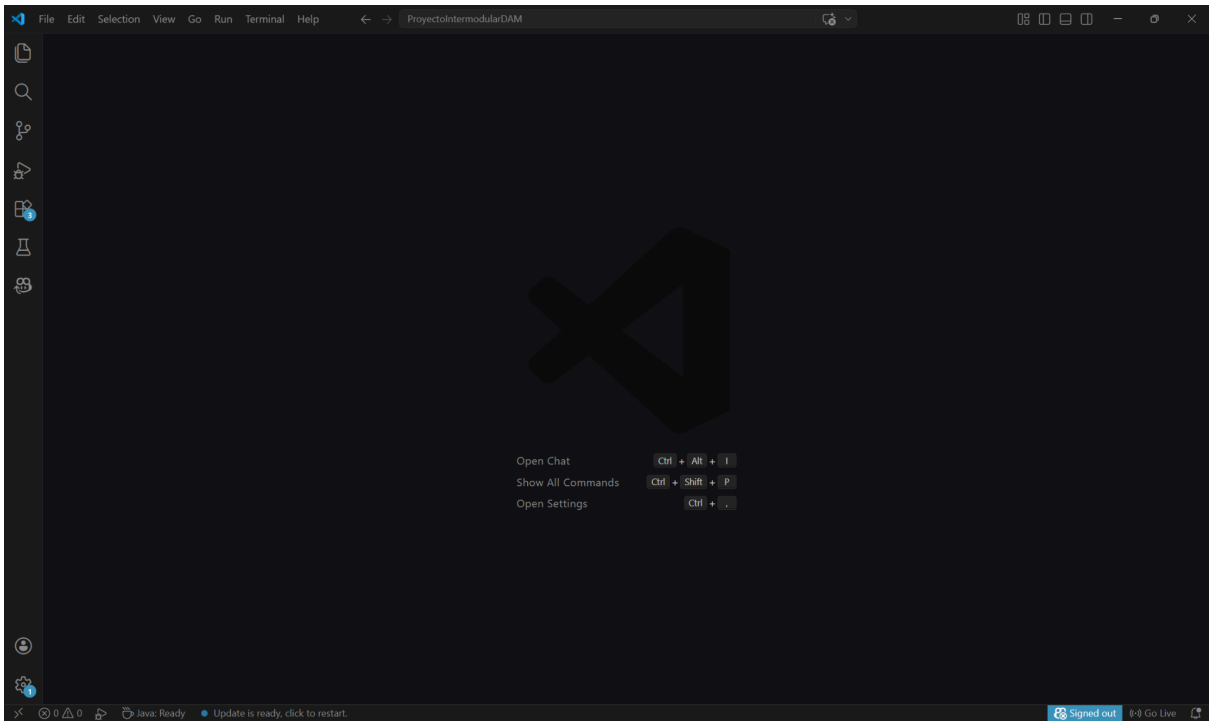
7.6 Visual Studio Code

Visual Studio Code es un editor de código fuente ligero, gratuito y multiplataforma que permite desarrollar aplicaciones en múltiples lenguajes de programación. Ofrece funcionalidades como resaltado de sintaxis, autocompletado, depuración y gestión de archivos, lo que facilita el desarrollo de software de forma eficiente.

En este proyecto, Visual Studio Code se ha utilizado como entorno de desarrollo principal para la implementación de la aplicación.

Se ha empleado para programar el código en Java, incluyendo la lógica del sistema, las clases DAO y la interfaz gráfica desarrollada con Swing. Además, también se ha utilizado para escribir y organizar los scripts SQL necesarios para la creación y gestión de la base de datos, los cuales posteriormente han sido ejecutados y gestionados en MySQL Workbench.

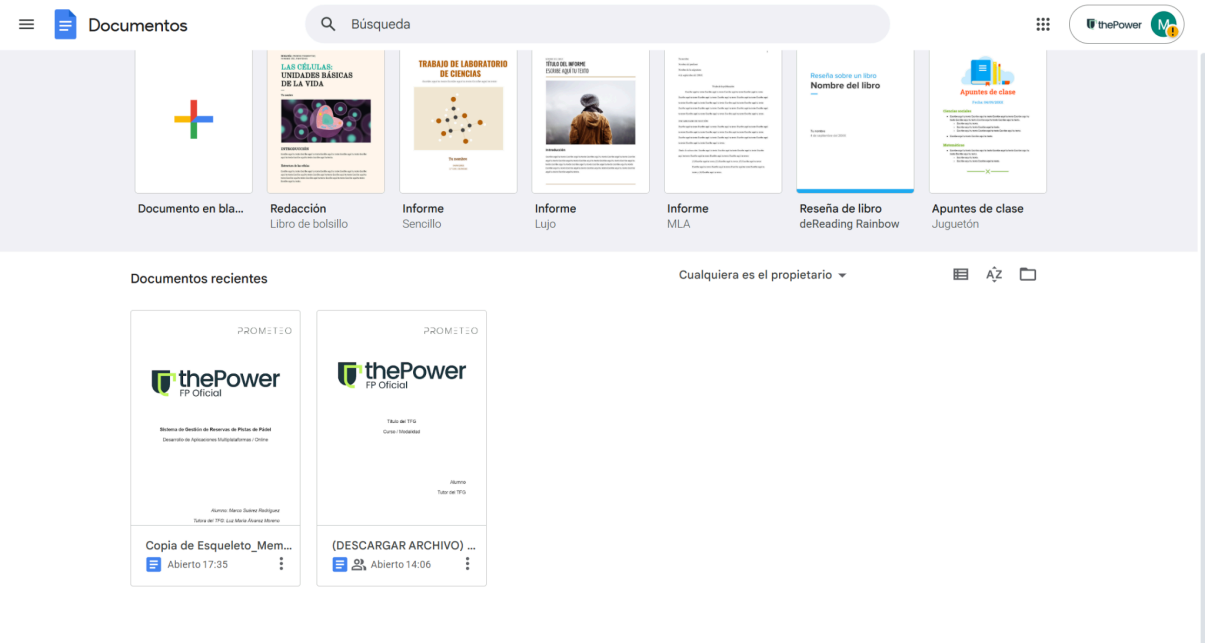
Gracias a esta herramienta, se ha podido centralizar el desarrollo del proyecto en un único entorno, facilitando la organización y el trabajo con distintos tipos de archivos.



7.7 Google Docs

Google Docs es una herramienta de edición de documentos en línea que permite crear, editar y compartir documentos de texto de forma colaborativa. Facilita la organización de la información y el acceso desde diferentes dispositivos.

En este proyecto, Google Docs se ha utilizado para la elaboración de la memoria y la documentación asociada al sistema. Ha permitido estructurar los distintos apartados del proyecto, redactar los contenidos y realizar modificaciones de forma sencilla. Además, facilita la exportación del documento a formatos como PDF para su entrega final.



8. METODOLOGÍA

8.1 Metodología Usada

Para el desarrollo del proyecto se ha seguido una metodología incremental, basada en la construcción progresiva del sistema mediante pequeñas fases funcionales.

En lugar de desarrollar el sistema completo de una sola vez, se ha optado por implementar primero la base de datos y las funcionalidades principales (CRUD), para posteriormente añadir mejoras como la lógica de negocio (cálculo automático de tarifas y validación de solapamientos) y, finalmente, la interfaz gráfica.

Este enfoque ha permitido comprobar el funcionamiento del sistema en cada fase, detectar errores de forma temprana y realizar correcciones de manera progresiva.

8.2 Justificación de la Metodología

Se ha elegido esta metodología debido a su simplicidad y adaptación a proyectos de tamaño medio como el presente.

El desarrollo incremental permite dividir el proyecto en partes más manejables, facilitando su comprensión y reduciendo la complejidad.

Además, este enfoque ha resultado especialmente útil al trabajar con múltiples tecnologías (Java, MySQL, JDBC y Swing), ya que ha permitido integrar cada una de ellas a su debido tiempo.

8.3 Planificación Inicial

Antes de comenzar el desarrollo del proyecto, se realizó una estimación del tiempo necesario para cada una de las fases principales, con el objetivo de organizar el trabajo de forma estructurada.

La planificación inicial fue la siguiente:

- ➔ Análisis: 1 semana
- ➔ Diseño de la Base de Datos: 1 semana
- ➔ Implementación DAO (CRUD): 2 semanas
- ➔ Lógica de Negocio: 1 semana
- ➔ Interfaz Gráfica: 2 semanas
- ➔ Pruebas: 1 semana
- ➔ Documentación: 1 semana

8.4 Planificación Real

El desarrollo real del proyecto ha seguido una planificación progresiva, en la que algunas tareas se han solapado en el tiempo.

La distribución final ha sido la siguiente:

- ➔ Análisis: Semana 1
- ➔ Diseño de la Base de Datos: Semana 2
- ➔ Implementación DAO (CRUD): Semanas 3,4 y 5
- ➔ Lógica de Negocio: Semana 6
- ➔ Interfaz Gráfica: Semana 6 y 7
- ➔ Pruebas: Semanas 2 a 7
- ➔ Documentación: Semana 3 a 7

En este caso, tareas como las pruebas y la documentación no se han realizado en una única fase, sino que se han llevado a cabo de forma continua a lo largo del desarrollo del proyecto.

8.5 Totalización del tiempo

El proyecto se ha desarrollado a lo largo de un total de 7 semanas, distribuyendo el trabajo entre las distintas fases según su complejidad.

Aunque algunas tareas se han solapado, la planificación ha permitido avanzar de forma progresiva, consolidando cada parte del sistema antes de continuar con la siguiente.

8.6 Análisis de Desviaciones

Al comparar la planificación inicial con la real, se observa que algunas fases han requerido más tiempo del previsto, especialmente la implementación de los DAO, debido a la complejidad de la conexión con la base de datos y la correcta implementación de las operaciones CRUD.

Asimismo, la realización de pruebas y la documentación se han llevado a cabo de forma continua durante varias semanas, en lugar de concentrarse en una única fase final como se había estimado inicialmente.

En general, la planificación ha sido adecuada, aunque ha sido necesario adaptarla a medida que avanzaba el desarrollo del proyecto, lo cual es habitual en este tipo de trabajos.

8.7 Diagrama de Gantt

El diagrama de Gantt se ha elaborado a partir de una estimación del tiempo dedicado a cada fase del proyecto dividido por semanas.

Debido a que no se ha realizado un registro exacto de horas durante el desarrollo, se ha reconstruido la planificación de manera coherente basándose en el trabajo realizado, los objetivos del proyecto y la complejidad de cada fase.

TAREAS/SEMANAS	S1	S2	S3	S4	S5	S6	S7	S8	S9
Análisis									
Base de Datos									
DAO									
Lógica									
Interfaz									
Pruebas									
Documentación									

9. TRABAJOS FUTUROS

El sistema desarrollado cumple con los objetivos planteados inicialmente, permitiendo la gestión de reservas de pistas de pádel mediante una aplicación funcional. No obstante, existen diversas líneas de mejora y ampliación que podrían implementarse en futuras versiones del proyecto.

En primer lugar, se podría mejorar la interfaz gráfica, incorporando componentes más avanzados como selectores de fecha (calendario) y hora, con el objetivo de facilitar la introducción de datos por parte del usuario y mejorar la experiencia de uso.

Otra posible mejora sería la automatización completa de ciertos parámetros, como el cálculo del tipo de día (laborable o fin de semana) y la franja horaria, evitando que el usuario tenga que seleccionarlos manualmente y reduciendo así posibles errores.

Asimismo, se podría ampliar el sistema incorporando nuevas funcionalidades, como la gestión de usuarios con autenticación (login), permitiendo diferenciar entre distintos tipos de usuarios (administradores y clientes).

También sería interesante añadir un sistema de pagos más completo, integrando métodos de pago reales o simulados, así como la generación de comprobantes o facturas.

Desde el punto de vista técnico, se podría evolucionar la aplicación hacia una arquitectura más avanzada, separando completamente las capas mediante el uso de patrones como MVC, o incluso migrando a una aplicación web utilizando tecnologías como HTML, CSS y JavaScript.

Por último, se podrían implementar mejoras en la base de datos, como la optimización de consultas, el uso de índices o la incorporación de nuevas tablas para ampliar la funcionalidad del sistema.

En conjunto, estas mejoras permitirían evolucionar el proyecto hacia una aplicación más completa, robusta y cercana a un entorno real de producción.

10. CONCLUSIONES

El desarrollo de este proyecto ha permitido aplicar de forma práctica los conocimientos adquiridos en el ciclo formativo, integrando diferentes tecnologías como Java, MySQL, JDBC y Java Swing en una única aplicación funcional.

A lo largo del proyecto, se ha diseñado e implementado un sistema completo de gestión de reservas de pistas de pádel, abarcando desde la modelización de la base de datos hasta la creación de una interfaz gráfica que permite al usuario interactuar con el sistema de manera sencilla.

Uno de los aspectos más relevantes ha sido la aplicación de la programación orientada a objetos y el uso del patrón DAO, lo que ha permitido estructurar el código de forma organizada y facilitar su mantenimiento. Además, se han incorporado funcionalidades adicionales como el cálculo automático de tarifas y la validación de solapamientos de reservas, aportando mayor realismo y robustez al sistema.

Durante el desarrollo, también se han afrontado diferentes dificultades, especialmente relacionadas con la gestión de la base de datos, la conexión mediante JDBC y la implementación de la lógica de negocio. Estas dificultades han sido resueltas de forma progresiva, lo que ha contribuido a reforzar la comprensión de los conceptos trabajados.

En conjunto, el proyecto cumple con los objetivos planteados inicialmente, ofreciendo una solución funcional y bien estructurada. Asimismo, sienta una base sólida para futuras ampliaciones y mejoras, como la evolución hacia aplicaciones más complejas o entornos web.

Este proyecto ha supuesto una experiencia clave para consolidar los conocimientos adquiridos y prepararse para desarrollos más complejos en el ámbito profesional.

REFERENCIAS

Según las normas APA.

Cada referencia se acompañará de un texto descriptivo con el apartado del proyecto asociado.

Formato:

Autor, A. A. (Año de publicación). Título de la página. Recuperado de URL

Ejemplo:

Aplicado en la investigación del tema de la web.

Smith, J. (2023). La importancia del reciclaje en la conservación del medio ambiente. Recuperado de https://www.ejemplodepagina.com/

Otro ejemplo:

Aplicado para realizar las vistas de la base de datos.

Oracle Corporation. (s. f.). Oracle Database 19c Documentation. Recuperado de https://docs.oracle.com/en/database/oracle/oracle-database/index.html